

## 4주 차 보고서

( 2026년 1 월 18일 ~ 2026년 1 월 24일 )

작성자

박신우

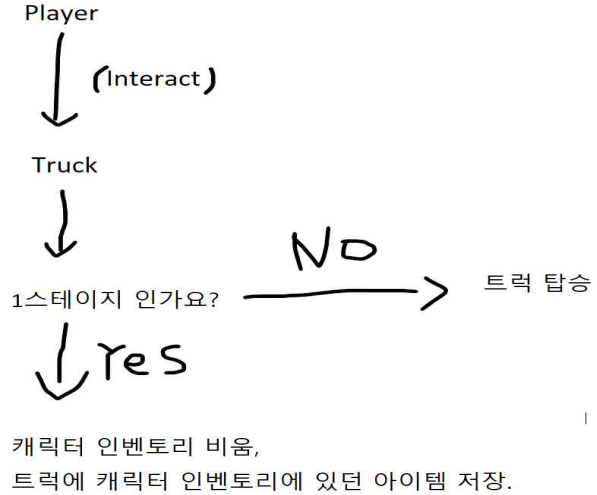
### 이번 주 한 일

#### 1.19 1스테이지 아이템 파밍 시스템 설계

##### 플레이어와 아이템의 상호작용



##### 플레이어와 트럭의 상호작용



#### 1.20 캐릭터 , 아이템 상호작용

지난주에 제작한 인터페이스 기반 상호작용 시스템 `IInteractInterface`과  
범위 기반 상호작용을 담당하는 `UInteractTriggerComponent`를 활용하여,  
플레이어가 게임 내에서 아이템을 루팅할 수 있도록 설계를 진행하였다.

이를 위해 1스테이지에서 루팅 가능한 다양한 아이템들의 공통 부모 클래스  
(`ALootItemBase`)를 새로 만들었다.

이 클래스는 상호작용 트리거 컴포넌트를 활용해,플레이어가 정해진 범위 내에 들어왔을  
때 상호작용 키 입력을 감지하여 아이템 획득이 가능하도록 하였다.

플레이어(캐릭터)는 이 구조를 바탕으로한 번에 최대 5개의 아이템을 인벤토리에 보관할 수  
있으며, 각 아이템은 상호작용을 통해 인벤토리에 추가된다.

```
LogTemp: Item Added. Current: 1
LogSlate: Took 0.013236 seconds to synchronously load lazily
LogTemp: Item Added. Current: 2
LogTemp: Item Added. Current: 3
LogTemp: Item Added. Current: 4
LogTemp: Item Added. Current: 5
LogTemp: Warning: Inventory Full!
LogTemp: Warning: Cannot pick up item: Inventory is full.
LogTemp: Warning: Inventory Full!
LogTemp: Warning: Cannot pick up item: Inventory is full.
LogTemp: Warning: Inventory Full!
LogTemp: Warning: Cannot pick up item: Inventory is full.
LogTemp: Warning: Inventory Full!
LogTemp: Warning: Cannot pick up item: Inventory is full.
LogTemp: Warning: Inventory Full!
LogTemp: Warning: Cannot pick up item: Inventory is full.
```

## 이번 주 한 일

### 1.21 캐릭터,트럭 상호작용

1스테이지에서는 캐릭터가 트럭과 상호작용할 때, 캐릭터 인벤토리의 모든 아이템을 트럭에 옮기고 인벤토리를 비우도록 구현하였다.

```
LogTemp: Warning: Inventory Full!  
LogTemp: Warning: Cannot pick up item: Inventory is full.  
LogTemp: Offloaded 5 items to Truck!
```

초기에는 캐릭터 인벤토리를 단순히 int형 변수(무게)로만 관리했으나, 이 경우 어떤 종류의 아이템을 들고 있었는지 트럭에서 알 수 없는 단점이 있었다.

이를 보완하기 위해, 각 아이템의 종류를 나타내는 열거형 EltemType의 배열(TArray<EltemType>)로 인벤토리 구조를 변경하였다.

이제는 트럭에 아이템을 전달할 때, 어떤 종류의 아이템을 옮겼는지도 정확히 알 수 있다.

아직 이 구조가 완벽하게 마음에 드는 것은 아니며, 공부를 해보며 추후 더 나은 아이디어가 떠오르면 개선할 계획이다.

### 1.22

여자친구 기념일이라 쉬었다..

### 1.23 트럭 Sound Cue 제작

Sound Cue는 언리얼 엔진에서 여러 사운드 소스와 오디오 처리를 하나로 조합,제어할 수 있는 오디오 에셋이다.

구매한 트럭 에셋에서 다양한 트럭 사운드가 있어서 이를 활용해서 트럭의 RPM에 따라 다른 사운드를 내도록 하려고 한다.

Sound Cue에 트럭이 빨리 움직일 때 , 천천히 움직일 때 idle 상태일 때 소리를 넣어서 트럭이 움직이면 소리가 나게 하려고 하는데 잘 작동하지 않았다.

### 1.24 트럭 사운드 문제 해결, 보고서 정리

Sound Cue에서 어떠한 RPM값에 소리를 틀 것인지 넣어주지 않았아서 발생한 문제였다. 값을 넣어주었더니 Idle 상태 소리는 잘 나는데 다른 상태에서의 소리가 잘 나지 않았다.

트럭 RPM에 따라 다른 사운드가 나도록 했는데 RPM쪽에서 문제가 있는 것 같다.

사운드 관련해서 문제를 찾아보니 보고서를 마무리 지어야 할 것 같아서. 보고서를 완성하였다.

#### 다음 주 할 일

트럭 사운드 관련 마무리.

현재 코드 공부하여 팀원들에게 설명

회의 진행하여 각자 진행상황 점검, 추가적인 에셋 검토.

좀비 기능.

#### 특이사항

이번 주에는 기념일 등 개인적인 일정이 있었고, 스스로도 다소 나태해졌음을 느꼈습니다.  
다음 주부터는 다시 한 번 마음을 다잡고, 더욱 집중할 계획입니다.

## 4주 차 보고서

( 2026년 1 월 18 일 ~ 2026년 1 월 24 일 )

작성자

안윤진

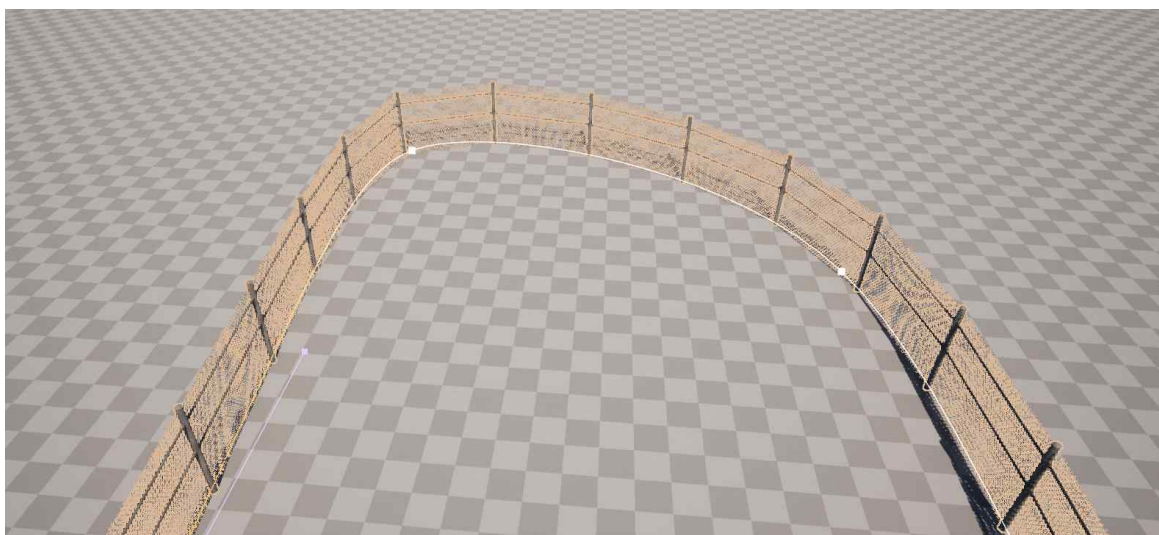
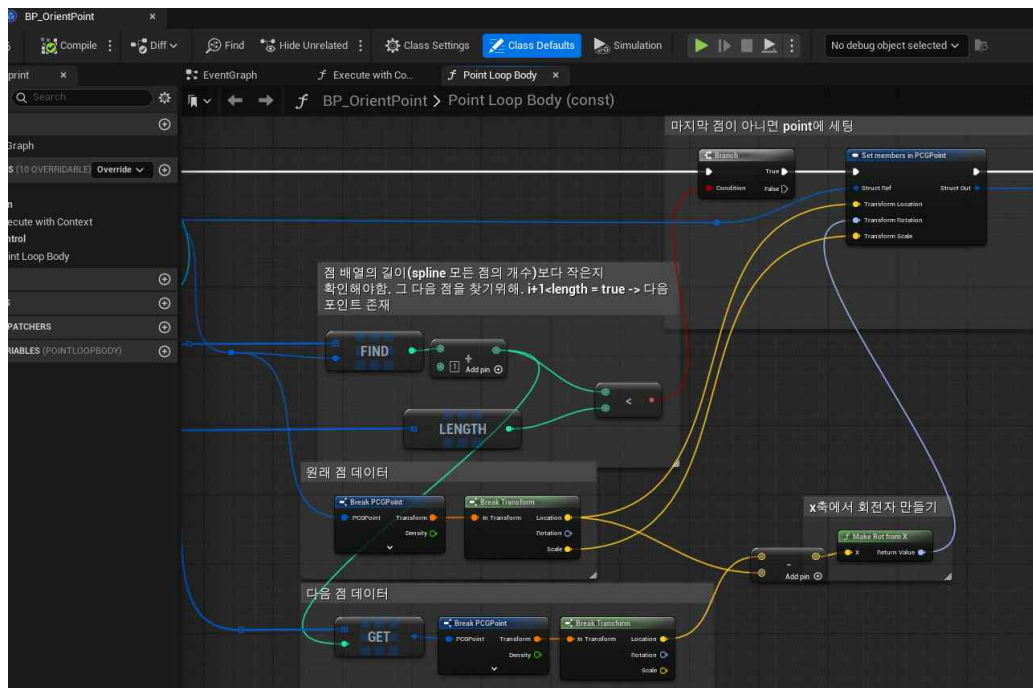
### 이번 주 한 일

pcg를 통한 집 제작 방식에 문제를 느껴 계획 변경하였다.

이 과정에서 맵에 대한 최적화에 대한 공부를 주로 해 결과물이 많지 않은 주였다.

pcg를 통한 제작은 길이나 자연물에 더 적합하다 판단해 맵 마을 테두리에 사용할 체인링크펜스를 제작.

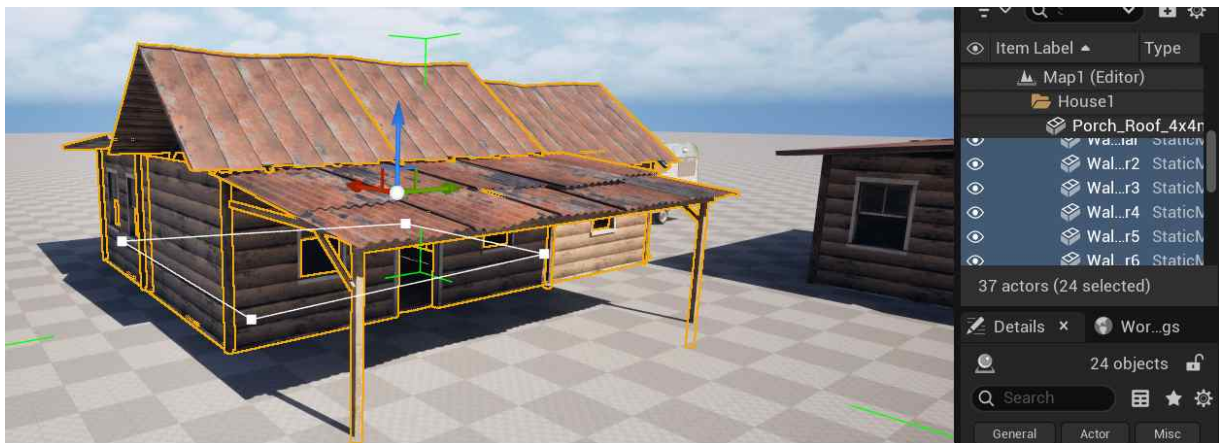
이때 블루프린트를 통해 다음 점과 연결되어 그려지게끔 수정. 내가 표현할 범위 내에서는 펜스들끼리 잘 이어지기에 이대로 진행.



## 이번 주 한 일



그리고 GPU VRAM 한도를 초과했다는 오류가 뜸. 그래픽 메모리가 다 차버리는 문제 발생.  
최적화 진행의 중요성 느낌. 이후 최적화에 고민하기 시작. (후에 알고보니 이건 노트북에서만 뜨던 문구였음,  
하지만 기왕 하는 거, 최적화 진행해보는 것이 훨씬 도움될 것이라 판단해 알아봄)



첫 번째 방법.

스태틱 메쉬들을 하나하나 배치해 집 구현 -> 메쉬들을 전부 합쳐 하나의 스태틱 메쉬로 변경



그러나 이후 hism/ism 에 대해 알게 되어 최적화 방향을 다시 변경하여 맵 제작 계획을 변경.

## 이번 주 한 일

1월 20일 화요일.

GPU < 명령 수행 시키는 주체: CPU.

CPU → GPU "여기에 이 객체 그려줘" → 드로우를

10000개의 나무 개별 액터 배치 시 → CPU 패프러임 10000번의 드로우를 보.

이때 명령 제때 주지 못해 GPU가 늦게됨 → CPU 병목 현상

→ 해결 가능할 기술. **HISM** Hierarchical Instanced Static Mesh

**ISM** (Instanced Static Mesh) ← 이 컴포넌트3 제공. (UE)

→ GPU 인스턴싱 → "이 나무 모델 이런 객체들에 10000개 그려줘" → 1번의 드로우.

ISM < 가장 기본적인 인스턴싱. 10000개의 인스턴스 → 하나의 거대 덩어리 취급.

∴ 단 1개라도 레벨에 오면 나머지 99,999개도 함께 인스턴싱 증가

GPU로 전송. → 메모리 사용량 문제 0.

해결 방법

ISM) 인스턴스들 (중간) 기준 여러 클러스터로 나눠 **제출 구조** 관리.

→ 사야할 클러스터 → 언리얼에서 제비.

음직임 X 인스턴싱에 적합.

PCG 소면 이미 사용하고 있는 것. 이 HISM이 기본 전체.

같은 바닥 매쉬 → 자동으로 하나의 HISM에 할당됨

바닥 종류 섞임 → FloorType 별로 HISM 분리.

Outliner → PCG 액터 확인. → ISM or HISM 있으면 잘라낸 것.

이 섹션은 액터 내부에서 진행.

※ 집을 전부 합쳐서 하나의 static mesh로 만드는 것 좋지 X.

⇒ 메모리 낭비. (같은 벽인데 집마다 다른 메시로 취급되게.)

| 차 | BP_집1      | BP_집2      | ...            |
|---|------------|------------|----------------|
|   | HISM floor | HISM floor | → 공통마다 HISM 따로 |
|   | HISM wall  | HISM ~     | draw call (번잡) |
|   | HISM ...   | 1, ~, ~    | ex) 이렇게는 나쁜 것  |

↑ ↑ 각각 다른 개체의 집 분포.

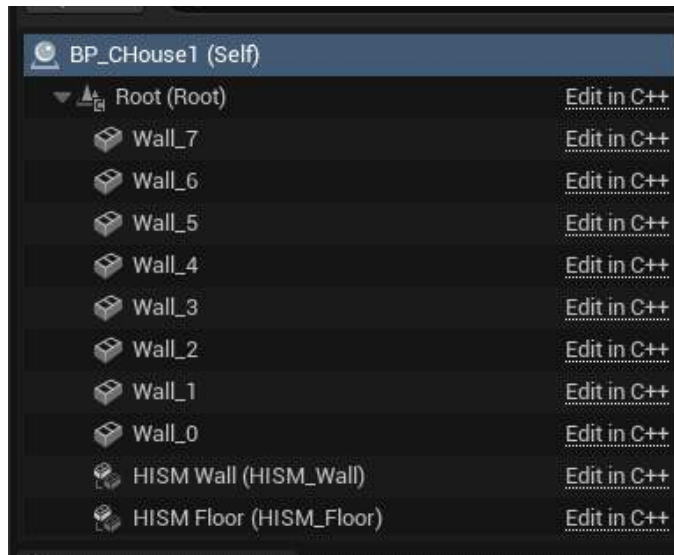
언리얼이 "같은 메시" 판단 기준. → 스테틱 메시 에셋 레퍼런스 같은

→ 같다면 다른 분포라도 전부 HISM으로 draw call 나옴.

언리얼은 메시가 (복사)가 아닌 (참조) 형태.

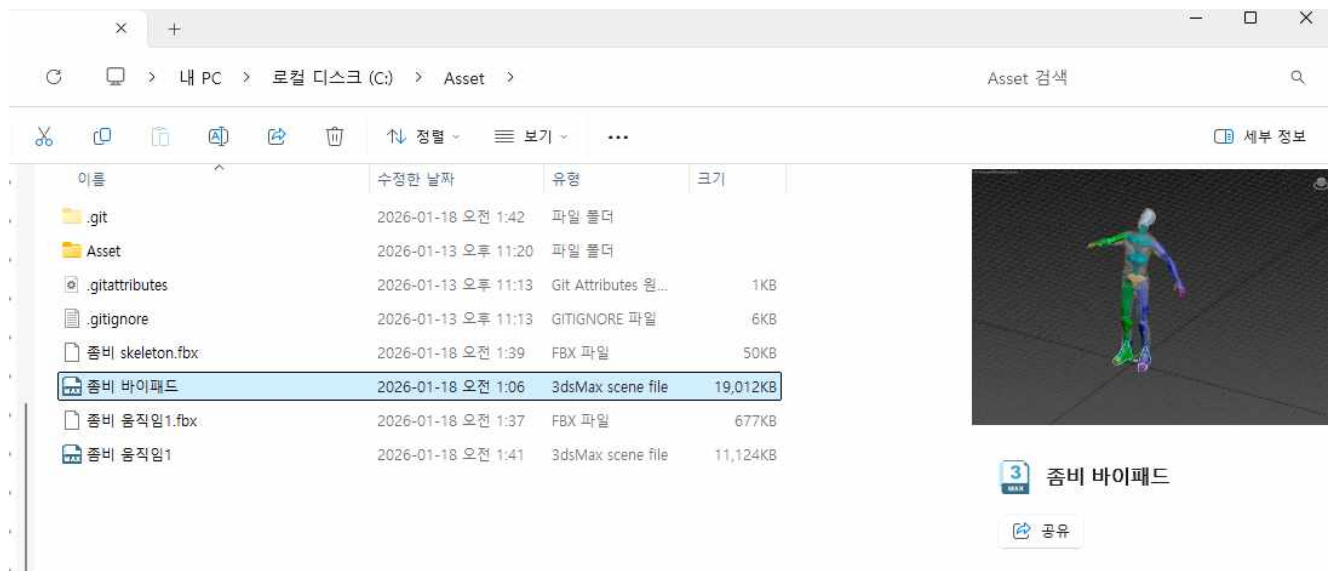
이 간단한 것들을 찾는 과정이 오래걸렸다.

집마다 각 액터로 제작, 이때 전체 레벨에서 반복되어 사용되는 메쉬들 (바닥이나 기둥)등은 hism으로 추가 하고, 나머지는 static mesh component로 추가해 액터를 제작하기로 계획.



현재 이렇게 class를 기반으로 컴포넌트들을 추가해, static mesh같은 경우에는 언리얼 에디터 상에서 수정 하고 hism같은 경우 class에서 instance를 통해 배치하며 집 액터 제작 중에 있음.

## 이번 주 한 일



기존 좀비 메시에 있는 스켈레톤 메시같은 형태로 애니메이션을 제작해 본 적이 없어, 좀비 바이패드를 따로 다시 제작해봄. 그리고 이 바이패드를 통해 기본 움직임 테스트 애니메이션을 제작해 넣어보려했는데, 이 스켈레톤 메시 연동 부분에서 문제 발생. 교체된 스켈레톤 메쉬를 적용하지 않고, 이 부분에서 공부가 너무 부족해 애니메이션 제작은 현재 미뤄둤.

시간 들이는 것이 아직 많이 부족하게 느껴짐. 좀 더 열심히 해야겠다 판단.

## 다음 주 할 일

집 에셋 4개이상 제작, 아이템 랜덤 배치& 전에 만들어둔 상호작용ui가져와 맵에 테스트 배치

## 특이사항

## 4주 차 보고서

( 2025년 1 월 18 일 ~ 2026년 1 월 24 일 )

작성자

배주환

### 이번 주 회의 내용

### 이번 주 한 일

LockFreeStack과 LockFreeQueue를 직접 만들어 보며 Lock-Free에 대한 이해도를 높였다. 아직 공부를 해도 잘 이해가 되지 않는 부분이라 나중에 적용해보면서 다시 한번 공부를 한다면 그때는 이해할 수 있을 것 같다.

```
void Push(const T& value)
{
    Node* node = new Node(value);
    node->next = _head;
    //if (_head == node->next)
    //{
    //    _head = node;
    //    return true;
    //}
    else
    {
        node->next = _head;
        return false;
    }
    while (_head.compare_exchange_weak(node->next, node) == false)
    {
    }
    // 이 사이에 새끼가 있냐?
    _head = node;
}

// 1) head 쓰기
// 2) head->next 쓰기
// 3) head = head->next
// 4) data 주 줄까지 읽음
// 5) 주 줄까지 쓰기
// [ 3 ] [ 1 ] [ 1 ] [ 1 ] [ 1 ]
// [head]
bool TryPop(T& value)
{
    Node* oldHead = _head;
    while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
    {
    }
    if (oldHead == nullptr)
    {
        return false;
    }
    value = oldHead->data;
    // 값이 삭제되면
    delete oldHead;
    return true;
}

bool TryPop(T& value)
{
    ++_popCount;
    Node* oldHead = _head;
    while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
    {
    }
    if (oldHead == nullptr)
    {
        --_popCount;
        return false;
    }
    value = oldHead->data;
    TryDelete(oldHead);
    return true;
}

// 1) 데이터 분리
// 2) Count 체크
// 3) 나 혼자만 삭제
void TryDelete(Node* oldHead)
{
    // 나 외에 누가 있는지를 체크
    if (_popCount == 1)
    {
        // 나 혼자네?
        // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
    }
}

void TryDelete(Node* oldHead)
{
    // 나 외에 누가 있는지를 체크
    if (_popCount == 1)
    {
        // 나 혼자네?
        // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
    }
}

template<typename T>
class LockFreeStack
{
public:
    struct Node
    {
        Node(const T& value) : data(value), next(nullptr)
        {
        }
        T data;
        Node* next;
    };

    void Push(const T& value)
    {
        Node* node = new Node(value);
        node->next = _head;
        while (_head.compare_exchange_weak(node->next, node) == false)
        {
        }
        _head = node;
    }

    bool TryPop(T& value)
    {
        ++_popCount;
        Node* oldHead = _head;
        while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
        {
        }
        if (oldHead == nullptr)
        {
            return false;
        }
        value = oldHead->data;
        TryDelete(oldHead);
        return true;
    }

    void TryDelete(Node* oldHead)
    {
        // 나 외에 누가 있는지를 체크
        if (_popCount == 1)
        {
            // 나 혼자네?
            // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
        }
        else if (node)
        {
            // 누가 끼어들었으니 다시 갖다 놓자
            ChainPendingNodeList(node);
        }
        // 내 데이터는 삭제
        delete oldHead;
    }
    else
    {
        // 누가 있네? 그럼 삭제하지 않고, 삭제 예약만
        ChainPendingNodeList(oldHead);
        --_popCount;
    }
}

void ChainPendingNodeList(Node* first, Node* last)
{
    last->next = _pendingList;
    while (_pendingList.compare_exchange_weak(last->next, first) == false)
    {
    }
}
```

```

void ChainPendingNodeList(Node* node)
{
    Node* last = node;
    while (last->next)
        last = last->next;

    ChainPendingNodeList(node, last);
}

void ChainPendingNode(Node* node)
{
    ChainPendingNodeList(node, node);
}

static void DeleteNodes(Node* node)
{
    while (node)
    {
        Node* next = node->next;
        delete node;
        node = next;
    }
}

private:
    atomic<Node*> _head;

    atomic<uint32> _popCount = 0; // Pop을 실행중인 스레드 개수
    atomic<Node*> _pendingList; // 삭제되어야 할 노드들(첫번째 노드)
};

shared_ptr<T> TryPop()
{
    CountedNodePtr oldHead = _head;
    while (true)
    {
        IncreaseHeadCount(oldHead);
        Node* ptr = oldHead.ptr;

        if (ptr == nullptr)
            return shared_ptr<T>();

        if (_head.compare_exchange_strong(oldHead, ptr->next))
        {
            shared_ptr<T> res;
            res.swap(ptr->data);

            // external : 1 -> 2(+1) -> 4(L+1, 남+2)
            // internal : 0

            // 나 알고 또 누가 읽는가?
            const int32 countIncrease = oldHead.externalCount - 2;
            if (ptr->internalCount.fetch_add(countIncrease) == -countIncrease)
                delete ptr;

            return res;
        }
        else if (ptr->internalCount.fetch_sub(1) == 1)
        {
            // 참조권은 얻었으나, 소유권은 실패 -> 뒷수습은 내가 한다
            delete ptr;
        }
    }
}

template<typename T>
class LockFreeQueue
{
public:
    struct Node;
    struct CountedNodePtr;
    struct NodeCounter;

    struct Node
    {
        NodeCounter newCounter;
        newCounter.internalCount = 0;
        newCounter.externalCountRemaining = 2;
        count.store(newCounter);

        next_ptr = nullptr;
        next_externalCount = 0;
    };

    void ReleaseRef()
    {
        NodeCounter oldCounter = count.load();
        while (true)
        {
            NodeCounter newCounter = oldCounter;
            newCounter.internalCount--;

            // 카운트를 줄 것임
            if (count.compare_exchange_strong(oldCounter, newCounter))
            {
                if (newCounter.internalCount == 0 && newCounter.externalCountRemaining == 0)
                    delete this;
                break;
            }
        }
    }

    shared_ptr<T> TryPop()
    {
        CountedNodePtr oldHead = _head.load();
        while (true)
        {
            // 참조권 획득 (externalCount를 현재값 기준 +1 한 애가 이길)
            IncreaseExternalCount(_head, oldHead);

            Node* ptr = oldHead.ptr;
            if (ptr == _tail.load()) ptr;
            ptr->ReleaseRef();
            return shared_ptr<T>();

            // 소유권 획득 (head = ptr->next)
            if (_head.compare_exchange_strong(oldHead, ptr->next))
            {
                T* res = ptr->data.exchange(nullptr);
                FreeExternalCount(oldHead);
                return shared_ptr<T>(res);
            }

            ptr->ReleaseRef();
        }
    }

    static void IncreaseExternalCount(atomic<CountedNodePtr*& counter, CountedNodePtr& oldCounter)
    {
        while (true)
        {
            CountedNodePtr newCounter = oldCounter;
            newCounter.externalCount++;

            if (counter.compare_exchange_strong(oldCounter, newCounter))
            {
                oldCounter.externalCount = newCounter.externalCount;
                break;
            }
        }
    }

    static void FreeExternalCount(CountedNodePtr& oldNodePtr)
    {
        Node* ptr = oldNodePtr.ptr;
        const int32 countIncrease = oldNodePtr.externalCount - 2;
        NodeCounter oldCounter = ptr->count.load();

        while (true)
        {
            NodeCounter newCounter = oldCounter;
            newCounter.externalCountRemaining--;
            newCounter.internalCount += countIncrease;

            if (ptr->count.compare_exchange_strong(oldCounter, newCounter))
            {
                if (newCounter.internalCount == 0 && newCounter.externalCountRemaining == 0)
                    delete ptr;
                break;
            }
        }
    }

private:
    atomic<CountedNodePtr*> _head;
    atomic<CountedNodePtr*> _tail;
};

template<typename T>
class LockFreeStack
{
public:
    struct Node;
    struct CountedNodePtr;

    struct Node
    {
        Node(const T& value) : data(make_shared<T>(value))
        {
            shared_ptr<T> data;
            atomic<int32> internalCount = 0;
            CountedNodePtr next;
        }
    };

    void Push(const T& value)
    {
        CountedNodePtr node;
        node.ptr = new Node(value);
        node.externalCount = 1;

        node.ptr->next = _head;
        while (_head.compare_exchange_weak(node.ptr->next, node) == false)
        {
        }
    }

private:
    atomic<CountedNodePtr*> _head;
};

```

그래도 이번주에 공부하면서 compare\_exchange\_strong이라는 함수에 대해서 제대로 이해를 했다. 멀티스레드 환경에서 스레드가 겹치는 일이 일어나지 않게 검증을 해주는 함수이다. expected와 desired라는 인자를 가지는데 만약에 들어온 값이 expected와 같다면 desired값과 같고 true를 리턴해주고, 다르면 들어온

값을 expected와 같은 값으로 두고 false를 리턴해준다.

이번주에는 서버 프레임워크에 ThreadManager를 추가해줬는데 쓰레드에게 번호를 1부터 지정해줘서 쓰레드마다 Id를 갖게 해주었고, join과 destroy를 하는 기능을 만들어주었다.

#### 다음 주 할 일

Lock에 대한 공부를 했으니 프레임워크에 Lock을 추가해주고 메모리에 대한 공부를 할 예정이다.  
그중에서 특히 스마트 포인터와 관련해서 메모리에 대해 깊이 탐구할 것이다.

#### 특이사항